

CMSC 104 Section 2

Spring 2026

Project 2

“Mastermind”

Date assigned:

Wednesday, March 4 2026

Date due:

Wednesday, March 25 2026

Assignment

Your assignment is to write a Python program that implements the guessing game, “Mastermind.” In this game, the computer will generate a random six-digit number. The user will be given up to 6 guesses to guess the number. After each guess, your program will tell the user whether the guess for each of the six digits is too high, too low or correct. In this way the user will be able to zero in on the correct answer.

Constraints:

The point of this program is for the student to get practice using Python functions, if-then statements, loops, and modular arithmetic. Therefore the following constraints are being imposed on the student’s solutions:

1. You must include at least four different functions, as described in the “Details” section below. You may include additional functions at your discretion, but you have to have at least the four described. Note: the four functions you write do not include the `break_out_numbers()` function we have given you below.
2. You must isolate each digit in the random number, and in each guess, using modular arithmetic and integer division.
3. **YOU MAY NOT USE LISTS!!** Lists are a fundamental Python data type that we will cover shortly after Spring Break. This program would be a lot easier to write if we let you

use lists. If you were to Google for a solution to this assignment, it will use lists. But don't do that. We haven't covered lists in class, and so you must solve this problem the longer, harder way.

Details

Here is what your program will do:

1. The first statement in your main program should be

```
from random import randint
```

This imports the routine to generate a random integer ("randint") using Python's pre-written random number generation library.

2. Call a function `print_welcome()`. This function prints out your welcome message to the user. It explains what the program does and how it works. This function has no parameters and no return statement.
3. Call a function `generate_number()`. This function calls the `randint` library that you previously imported. You must generate a six digit number that is between 100000 and 999999, inclusive. This function does not need a parameter, but must return the generated number. This is the number that the user must guess.
4. Now comes the tricky part. Include this function in your code:

```
def break_out_numbers(num):  
    n6 = num % 10  
    n5 = (num // 10) % 10  
    n4 = (num // 100) % 10  
    n3 = (num // 1000) % 10  
    n2 = (num // 10000) % 10  
    n1 = (num // 100000) % 10  
    return n1, n2, n3, n4, n5, n6
```

Here's what this function does. It takes the six-digit number and returns the six digits, in order, as six one-digit numbers.

Call this function with the parameter being the number you generated in step 3. In order to get the six digits back, your call will look something like this:

```
x1, x2, x3, x4, x5, x6 = break_out_numbers(n)
```

where "n" is the number you generated, and x1 is the first digit of that number; x2 is the second digit; x3 is third; x4 is fourth; x5 is fifth and x6 is the last digit - the ones digit, if you will.

5. Now, in your main program, set the number of guesses to be 0. Start a loop that will run up to 6 tries. We recommend a while loop, but you can use a for loop if you're more comfortable with it.
 - a. Within the loop, increment the guess counter by one.
 - b. all a function, `get_guess()` that prompts the user to input a six-digit integer between 100000 and 999999 inclusive. This function will have no parameters; it will return the user's guess.

- c. Now, call the `break_out_numbers()` function with the guess as the parameter. The call will be similar to the one in step 4 - but use different variables!!! You will now have the six digits of the user's guess, in order.
- d. Next, compare each digit in the guess to the corresponding digit in the generated random number. If the guessed digit is too high, print out a message saying this digit's guess is too high. If the guessed digit is too low, print out a message saying this digit's guess is too low. If the guessed digit is correct, print out a message saying this digit's guess is correct

NOTE: if we let you use lists, you could just write a loop to do the six comparisons above. But you're not allowed to use lists. So you're going to have to write six different if-elif-else statements. Sorry, but that's where we are in the class.

- e. If all six guessed digits are correct, you may exit the loop and proceed to the last function. Otherwise, you start the loop again.
6. Finally, write a function that prints out a summary at the end of the game. If the user correctly guessed each digit of the number, you should print out a message saying something like "congratulations. You have guessed the number in Y guesses" where you replace Y with the actual number of guesses it took. If the user has used all six guesses and still not guessed the number, you should print out a message saying "I'm sorry, you are not the mastermind this time. You are welcome to try again."

Grading Rubric:

At least four functions, not including `break_out_numbers()` - 25 points

Correct operation when the user guesses the number - 25 points

Correct operation when the user has used all six guesses and failed to guess the number - 25 points

Proper use of comments in the program - 10 points

Good variable names; indentation, and other style points - 15 points.